РЕШЕН ИЗПИТ С ПЪЛНИ ОБЯСНЕНИЯ

Базови Програмни Езици — C (plain C)

Изпит 5: Фитнес абонаменти | 100 точки

Защо точно този изпит?

Изпит 5 покрива ТРИТЕ класически шаблона от програмата:

struct + realloc + текстов файл | средна стойност + филтър | бинарно търсене

Тази структура се среща в 5 от 6 анализирани изпита.

СЕКЦИЯ 0 — ПРОЧИТ И АНАЛИЗ НА ЗАДАЧАТА

Преди да напишем и един ред код, анализираме задачата методично. Тази стъпка е задължителна на изпита — ако я пропуснеш, може да напишеш грешна структура или да ползваш грешен файлов режим.

Задача 1 — как декодираме условието

КОД / ИЗРАЗ	КАКВО ОЗНАЧАВА ЗА НАС
`статичен низ, дължина 55`	char names[55]; — НЕ char*, никога char* в struct!
`формат "AA1234" (6 символа)`	char id[7]; — 6 символа + 1 за нулевия терминатор \0
`реално число, формат 0.00`	float price; — при printf: %.2f
`цяло трицифрено число`	int locker; — обикновен int, без специален формат
`динамично разширява масива`	realloc() шаблон — задължителна Задача 1 конструкция
`записва в txt с разделител ';'`	fprintf(f, "%d;%s;%s;%.2f;%d\n", ...) — fopen("a")

Задача 2 — какво се иска

Изчисли средноаритметичната цена на всички членове, после принтирай само тези, чиято цена е СТРОГО по-малка от нея. Изисква два отделни цикъла — не един.

Задача 3 — 40 точки, най-важна

Приема ID (формат AA1234), чете от бинарен файл "members.bin" и принтира данните за намерения член. Ключовото: fopen с "rb", четене с fread в while цикъл, сравнение с strcmp — НИКОГА с ==.

СЕКЦИЯ 1 — ПЪЛЕН КОД (компилируем: gcc -Wall -std=c99)

> **ВАЖНО:** Прочети кода от началото до края, преди да четеш обясненията. Опитай се да предположиш какво прави всяка функция.

```c
/* ================================================================ */
/*  ФИТНЕС АБОНАМЕНТИ — Пълно решение                               */
/*  Задача 1 (30т) + Задача 2 (30т) + Задача 3 (40т)                 */
/* ================================================================ */

#include <stdio.h>    /* printf, scanf, fprintf, fscanf, fopen, fclose */
#include <stdlib.h>   /* realloc, free, exit */
#include <string.h>   /* strcpy, strcmp, strlen */

/* ── СТРУКТУРАТА ──────────────────────────────────── */
typedef struct {
    char  names[55];  /* до 54 символа + '\0' — НИКОГА char* */
    char  id[7];      /* AA1234 = 6 символа + '\0'           */
    float price;      /* реално число: 13.50                 */
    int   locker;     /* цяло число:   579                   */
} Member;

/* ── ЗАДАЧА 1: addMember() ──────────────────────────── */
Member* addMember(Member *arr, int *count) {
    (*count)++;                  /* увеличи брояча чрез указател */

    arr = (Member*) realloc(arr, (*count) * sizeof(Member));
    if (arr == NULL) {           /* ЗАДЪЛЖИТЕЛНА проверка! */
        fprintf(stderr, "realloc failed!\n");
        exit(1);
    }

    Member *m = &arr[(*count) - 1];  /* указател към последния елемент */
    printf("  Names : "); scanf("%54s", m->names);
    printf("  ID    : "); scanf("%6s",  m->id);
    printf("  Price : "); scanf("%f",  &m->price);
    printf("  Locker: "); scanf("%d",  &m->locker);

    FILE *f = fopen("membersText.txt", "a"); /* "a" = добавяй */
    if (f != NULL) {
        fprintf(f, "%d;%s;%s;%.2f;%d\n",
                (int)strlen(m->names),
                m->names, m->id, m->price, m->locker);
        fclose(f);  /* ЗАДЪЛЖИТЕЛНО */
    }

    return arr;  /* ЗАДЪЛЖИТЕЛНО — realloc може да е преместил масива! */
}

/* ── ЗАДАЧА 2: filterBelowAverage() ──────────────── */
void filterBelowAverage(Member *arr, int count) {
    if (count == 0) return;  /* предпазна проверка */

    float sum = 0;
    for (int i = 0; i < count; i++)
```

```c
        sum += arr[i].price;
    float avg = sum / count;

    printf("Средна стойност: %.2f\n", avg);
    for (int i = 0; i < count; i++)
        if (arr[i].price < avg)
            printf("%s - %s - %.2f\n",
                   arr[i].names, arr[i].id, arr[i].price);
}

/* ── ЗАДАЧА 3: saveBinary() + searchBinary() ──────── */
void saveBinary(Member *arr, int count) {
    FILE *f = fopen("members.bin", "wb"); /* 'b' е ЗАДЪЛЖИТЕЛНО! */
    if (f == NULL) { printf("Грешка!\n"); return; }
    fwrite(arr, sizeof(Member), count, f);
    fclose(f);  /* ЗАДЪЛЖИТЕЛНО */
}

void searchBinary(const char *searchId) {
    FILE *f = fopen("members.bin", "rb"); /* 'b' е ЗАДЪЛЖИТЕЛНО! */
    if (f == NULL) { printf("Файлът не съществува!\n"); return; }

    Member m;
    int found = 0;
    while (fread(&m, sizeof(Member), 1, f) == 1) {  /* чети по 1 запис */
        if (strcmp(m.id, searchId) == 0) {  /* НИКОГА m.id == searchId */
            printf("Bin Names: %s\n",    m.names);
            printf("Bin ID: %s\n",       m.id);
            printf("Bin M_Price: %.2f\n", m.price);
            printf("Bin Locker: %d\n",   m.locker);
            found = 1;
            break;  /* намерихме — спираме */
        }
    }
    if (!found)
        printf("Член с ID '%s' не е намерен.\n", searchId);
    fclose(f);  /* ЗАДЪЛЖИТЕЛНО */
}

/* ── MAIN ──────────────────────────────────────── */
int main() {
    Member *arr = NULL;  /* NULL = масивът е празен */
    int count = 0;

    printf("=== Задача 1: Добавяне ===\n");
    arr = addMember(arr, &count);  /* ЗАДЪЛЖИТЕЛНО arr = ... */
    arr = addMember(arr, &count);
    arr = addMember(arr, &count);

    printf("\n=== Задача 2: Под средното ===\n");
    filterBelowAverage(arr, count);

    printf("\n=== Задача 3: Бинарно търсене ===\n");
    saveBinary(arr, count);
    char searchId[7];
    printf("Въведи ID: "); scanf("%6s", searchId);
    searchBinary(searchId);

    free(arr);  /* ЗАДЪЛЖИТЕЛНО */
    return 0;
}
```

СЕКЦИЯ 2 — ЗАДАЧА 1: СТРУКТУРАТА (typedef struct)

Ред по ред: декларацията

КОД	ОБЯСНЕНИЕ
`typedef struct {`	Казваме: 'дефинираме нов потребителски тип'. typedef означава, че след } ще дадем кратко название — после пишем само Member, не struct Member навсякъде.
`char names[55];`	Поле за двете имена. 55 = 54 символа + 1 за нулевия терминатор '\0'. НИКОГА char *names — при fwrite/fread char* записва само адреса, не текста!
`char id[7];`	Поле за ID с формат AA1234. AA1234 = 6 символа, затова 6+1=7. Формулата: char field[максимум_символа + 1].
`float price;`	Реалното число за цената. Задачата казва 'формат 0.00' — при printf ползваме %.2f (2 знака след точката).
`int locker;`	Целото число за шкафчето. Задачата казва 'трицифрено', но в C това е просто int — няма специален тип.
`} Member;`	Затваряме блока и даваме името Member. От тук: Member m; Member *arr; Member arr[100]; — без думата struct.

ВАЖНО: В паметта Member заема sizeof(Member) байта — обикновено около 70 байта. ЗАДЪЛЖИТЕЛНО ползвай sizeof(Member) навсякъде: в realloc, fwrite, fread. Никога не пиши фиксирано число!

Защо char[] и НИКОГА char* в структура?

Това е най-честата грешка. При fwrite записваме блока от памет директно на диска. Ако имаш char*, в структурата се пази само адрес (8 байта) — при запис на диска ще запишем само адреса, не самия текст. При следващо fread ще прочетем стар адрес, вече невалиден — програмата се срива.

```
[NE]  char *names;       // пази само АДРЕС (8 байта). fwrite ще запише адреса, не текста!
[OK]  char names[55];    // 55 байта ДИРЕКТНО в структурата — fwrite работи правилно
```

Ред по ред: функцията addMember()

КОД	ОБЯСНЕНИЕ
`Member* addMember(`	Типът на връщане е Member* — указател. Функцията ТРЯБВА да върне новия адрес, защото realloc може да е преместил масива!
`  Member *arr, int *count)`	arr е масивът. count е указател — за да модифицираме брояча в main(). Ако беше int count (без *), промените щяха да се губят при return.
`  (*count)++;`	(*count) разкожва указателя — стигаме до оригиналната count в main(). Без скоби: *count++ увеличава самия адрес (pointer arithmetic), не стойността — bug!
`  arr = (Member*) realloc(arr,`	ЗАДЪЛЖИТЕЛНО arr = realloc(...). Без arr = губим новия адрес. (Member*) е явно преобразуване на типа (cast).
`  (*count) * sizeof(Member));`	Новият размер: брой-елементи * размер-на-един-елемент. sizeof(Member) дава точния брой байта — включително padding.
`  if (arr == NULL) { exit(1); }`	realloc връща NULL ако паметта е свършила. Без тази проверка ще пишем в адрес 0 — програмата се срива. На изпита тази проверка носи точки!
`  Member *m = &arr[(*count)-1];`	Взимаме адреса на ПОСЛЕДНИЯ елемент (index = count-1). По-удобно от arr[(*count)-1].names навсякъде — пишем m->names.
`  scanf("%54s", m->names);`	БЕЗ & пред m->names — names е масив, масивът вече е адрес! %54s = максимум 54 символа.
`  scanf("%f", &m->price);`	ЗА числа (float, int) задължително &. m->price е стойност, не адрес — трябва & за 'запиши ТУК'.
`  FILE *f = fopen("...", "a");`	"a" = append (добавяне). Всяко извикване добавя нов ред. Ако беше "w", всеки път щеше да изтрива файла!
`  fprintf(f, "%d;%s;%s;%.2f;%d\n",`	Записваме в текстовия файл. Форматът е точно от условието: дължина;names;id;цена;шкафче с разделител ';'.
`  (int)strlen(m->names),`	strlen() брои символите БЕЗ '\0'. (int) е cast, защото strlen връща size_t (беззнаков тип), а %d очаква int.
`  fclose(f);`	ЗАДЪЛЖИТЕЛНО след всяка работа с файл. Без fclose() данните може да са в буфер и никога да не стигнат до диска!
`  return arr;`	ЗАДЪЛЖИТЕЛНО! Caller-ът трябва да получи новия адрес. В main() ЗАДЪЛЖИТЕЛНО: arr = addMember(arr, &count);

СЕКЦИЯ 3 — ЗАДАЧА 2: СРЕДНА СТОЙНОСТ И ФИЛТРИРАНЕ

Задача 2 почти винаги изисква: изчисли нещо (средна стойност, брой, сума) и принтирай онези елементи, които отговарят на условие. Алгоритъмът е фиксиран и се учи наизуст.

Защо два цикъла, а не един?

Честата грешка е да се опита всичко в един цикъл. Не е възможно: за да знаем дали цената е 'под средната', трябва ПЪРВО да сметнем средната, а тя се знае само СЛЕД като обходим всички елементи.

КОД	ОБЯСНЕНИЕ
`if (count == 0) return;`	Предпазна проверка. Ако масивът е празен — sum/count е деление на нула в C — undefined behavior. Винаги проверявай!
`float sum = 0;`	Инициализираме с нула. Ако пропуснеш — sum ще съдържа случайно число от паметта (т.нар. 'garbage value').
`for (int i = 0; i < count; i++)`	i започва от 0 (не 1!). i < count (не i <= count — ще излезем извън масива!). Стандартна C итерация.
`sum += arr[i].price;`	Добавяме цената на i-тия член. arr[i] е елементът на позиция i. .price достъпва полето чрез точка (arr[i] е обект, не указател).
`float avg = sum / count;`	Средна стойност = сума / брой. Работи защото sum е float — деленето е с плаваща запетая. Ако sum беше int, резултатът щеше да е цяло число!
`for (int i = 0; i < count; i++)`	ВТОРИ цикъл — сега вече знаем avg и можем да проверяваме всеки елемент срещу нея.
`if (arr[i].price < avg)`	Условието от задачата: СТРОГО по-малко (<), не по-малко или равно (<=). Прочитай условието внимателно!
`printf("%s - %s - %.2f\n",`	Форматът е точно от условието: 'Две Имена - ID - Сума'. %.2f = 2 знака след десетичната точка.

ВАЖНО: Ако задачата иска 'намали цената с 20%' вместо 'принтирай' — пишеш: arr[i].price *= 0.80; Ако иска 'брои' вместо 'принтирай' — пишеш: result++ и return result. Логиката е същата, само действието се мени.

СЕКЦИЯ 4 — ЗАДАЧА 3: БИНАРНИ ФАЙЛОВЕ (40 точки)

Тази задача носи 40 точки — повече от всяка друга. Важно е да разбереш как работят fwrite и fread, защото типичните грешки (липсващото 'b', == вместо strcmp) коват 10-15 точки.

Бинарен срещу текстов файл — каква е разликата?

В текстовия файл числото 13.50 се записва като 5 символа: '1', '3', '.', '5', '0' — точно като в Notepad. В бинарния файл 13.50 се записва като 4 байта в IEEE 754 формат — не е четимо с Notepad. Предимството: fwrite копира целия struct директно от паметта на диска. fread го прочита обратно — никакво парсване, никакви scanf формати.

Синтаксис на fwrite и fread

```
/* fwrite(pointer, размер_на_елемент, брой_елементи, файл)          */
/* Връща: брой записани елементи — трябва да е == count              */
size_t fwrite(const void *ptr, size_t size, size_t count, FILE *f);

/* fread(pointer, размер_на_елемент, брой_елементи, файл)           */
/* Връща: брой прочетени елементи — 0 означава EOF или грешка        */
size_t fread(void *ptr, size_t size, size_t count, FILE *f);

/* Запис на ЕДИН елемент: */
fwrite(&m, sizeof(Member), 1, f);

/* Запис на ЦЕЛИЯ масив: */
fwrite(arr, sizeof(Member), count, f);  /* arr, не &arr! */
```

Ред по ред: saveBinary()

КОД	ОБЯСНЕНИЕ
`fopen("members.bin", "wb")`	'w' = write (запис от нула). 'b' = binary — ЗАДЪЛЖИТЕЛНО. Без 'b' на Windows всеки байт 0x0A се превръща в 0x0D 0x0A и бинарните данни се разрушават.
`if (f == NULL) { return; }`	Задължителна проверка. fopen връща NULL ако не може да създаде файла — няма права, дискът е пълен и т.н.
`fwrite(arr,`	Какво записваме: адресът на масива. arr (без &) е вече адрес на първия елемент. &arr е адрес на указателя — грешно!
`sizeof(Member),`	Колко байта е ЕДИН елемент. sizeof(Member) дава точния брой — включително padding между полетата.
`count, f);`	Колко елемента и в кой файл. fwrite записва count * sizeof(Member) байта последователно на диска.

КОД	ОБЯСНЕНИЕ
`fclose(f);`	ЗАДЪЛЖИТЕЛНО. I/O в C е буферирано — данните може да са в RAM. fclose принуждава записа на диска.

Ред по ред: searchBinary()

КОД	ОБЯСНЕНИЕ
`fopen("members.bin", "rb")`	'r' = read. 'b' = binary — ЗАДЪЛЖИТЕЛНО и при четене.
`Member m;`	Буфер за ЕДИН запис. fread ще чете директно в тази структура в паметта.
`int found = 0;`	Флаг 'намерен ли е'. Ако след цикъла found == 0 — принтираме 'не е намерен'.
`while (fread(&m, sizeof(Member), 1, f) == 1)`	Четем по ЕДИН запис. fread връща броя прочетени елементи. При 1 = успех. При 0 = EOF или грешка — излизаме от цикъла.
`if (strcmp(m.id, searchId) == 0)`	strcmp сравнява СЪДЪРЖАНИЕТО на два низа. Връща 0 ако са равни. НИКОГА if (m.id == searchId) — това сравнява АДРЕСИ, не текст!
`found = 1; break;`	Намерихме — вдигаме флага и излизаме. Без break щяхме да четем до края на файла без нужда.
`if (!found) printf("Не е намерен");`	Ако found е все още 0, обходихме целия файл без съвпадение.
`fclose(f);`	ЗАДЪЛЖИТЕЛНО — дори при четене. fclose освобождава FILE структурата от паметта.

СЕКЦИЯ 5 — ИЗПЪЛНЕНИЕ СТЪПКА ПО СТЪПКА

Проследяваме с три примерни члена: Ivan Ivanov (AA1234, 13.50лв, шкафче 579), Pesho Petrov (AB1233, 15.00лв, 102) и Maria Georgieva (BB1333, 50.00лв, 321).

Стъпка 1 — main() стартира

```
Member *arr = NULL;   // arr сочи към НИЩО — NULL е валидно за realloc!
int count = 0;        // масивът е празен
// В паметта: arr = 0x00000000 (NULL),  count = 0
```

Стъпка 2 — arr = addMember(arr, &count) [ПЪРВИ ЧЛЕН]

```
// Влизаме в addMember(NULL, &count)
(*count)++;                  // count: 0 -> 1
arr = realloc(NULL, 1 * sizeof(Member));  // = malloc — заделя ~70 байта
// arr сега сочи към нов блок: напр. адрес 0x7F1000
Member *m = &arr[0];         // m сочи към arr[0]
// Потребителят въвежда: IvanIvanov  AA1234  13.50  579
// Запис в membersText.txt: 10;IvanIvanov;AA1234;13.50;579
return arr;                  // връщаме 0x7F1000 на main()
```

Стъпка 3 — arr = addMember(arr, &count) [ВТОРИ ЧЛЕН]

```
// count е вече 1. Влизаме в addMember(0x7F1000, &count)
(*count)++;                  // count: 1 -> 2
arr = realloc(0x7F1000, 2 * sizeof(Member));
// realloc МОЖЕ да върне РАЗЛИЧЕН адрес: напр. 0x7F2000 !
// Ако не бяхме върнали arr, main() щеше да работи с 0x7F1000 — вече освободена памет!
Member *m = &arr[1];         // m сочи към arr[1]
// Потребителят въвежда: PeshoPetrov  AB1233  15.00  102
return arr;                  // връщаме 0x7F2000 (новия адрес!)
```

Стъпка 4 — filterBelowAverage(arr, 3)

```
// arr[0].price = 13.50,  arr[1].price = 15.00,  arr[2].price = 50.00
sum = 13.50 + 15.00 + 50.00 = 78.50
avg = 78.50 / 3 = 26.17

// Втори цикъл — проверяваме всеки:
arr[0].price = 13.50  <  26.17  ->  ПРИНТИРА: Ivan Ivanov - AA1234 - 13.50
arr[1].price = 15.00  <  26.17  ->  ПРИНТИРА: Pesho Petrov - AB1233 - 15.00
arr[2].price = 50.00  >  26.17  ->  не принтира
```

Стъпка 5 — saveBinary(arr, 3) + searchBinary("AA1234")

```
// saveBinary: записва 3 * sizeof(Member) байта в members.bin
fwrite(arr, sizeof(Member), 3, f);
// members.bin: [Ivan ~70б][Pesho ~70б][Maria ~70б]  (RAW байтове)

// searchBinary("AA1234"):
// Итерация 1: fread чете ~70 байта -> m = Ivan Ivanov,  m.id = "AA1234"
// strcmp("AA1234", "AA1234") == 0  ->  НАМЕРЕН!
// Принтира: Bin Names: Ivan Ivanov
//           Bin ID: AA1234
//           Bin M_Price: 13.50
```

```
//             Bin Locker: 579
// break -> излизаме без да четем останалите записи
```

СЕКЦИЯ 6 — ТОП 10 ГРЕШКИ НА ИЗПИТА

Тези грешки се повтарят в почти всяко решение на начинаещи. Прочети, запомни, и преди предаване провери всяка.

ГРЕШКА 1 — char* вместо char[] в структура

```
[NE]  char *names;                    // само адрес (8 байта) — fwrite/fread не работят!
[OK]  char  names[55];                // 55 байта директно в struct — fwrite/fread OK
```

ГРЕШКА 2 — Присвояване на низ с =

```
[NE]  m.names = "Ivan";               // масив не може да се присвоява с = !
[OK]  strcpy(m.names, "Ivan");        // strcpy копира байт по байт — правилно
```

ГРЕШКА 3 — Сравнение на низове с ==

```
[NE]  if (m.id == searchId)           // сравнява АДРЕСИ, не съдържание!
[OK]  if (strcmp(m.id, searchId) == 0)  // сравнява СЪДЪРЖАНИЕТО
```

ГРЕШКА 4 — realloc без присвояване

```
[NE]  realloc(arr, n * sizeof(Member));        // arr НЕ се обновява — старият адрес!
[OK]  arr = realloc(arr, n * sizeof(Member));  // arr сочи към новия блок
```

ГРЕШКА 5 — void addMember() без return arr

```
[NE]  void addMember(Member *arr, int *count) { ... }    // caller не вижда новия arr!
[OK]  Member* addMember(Member *arr, int *count) { ... return arr; }
```

ГРЕШКА 6 — fopen без 'b' за бинарни файлове

```
[NE]  fopen("data.bin", "r")    // 'r' е за текстови — бинарните данни се разрушават!
[OK]  fopen("data.bin", "rb")   // 'b' е задължително
```

ГРЕШКА 7 — Без NULL проверка след fopen

```
[NE]  FILE *f = fopen("data.bin", "rb"); fread(&m, ...);   // CRASH ако f == NULL!
[OK]  FILE *f = fopen(...); if (!f) { return; } fread(...);  // безопасно
```

ГРЕШКА 8 — Без fclose()

```
[NE]  fwrite(arr, ...);   /* fclose липсва */    // данните може да не са на диска!
[OK]  fwrite(arr, ...); fclose(f);               // гарантирано записано
```

ГРЕШКА 9 — count++ вместо (*count)++

```
[NE]  count++;              // увеличава ЛОКАЛНОТО копие — main() не го вижда!
[OK]  (*count)++;           // увеличава оригиналния count в main()
```

ГРЕШКА 10 — char id[6] за формат AA1234

```
[NE]  char id[6];    // AA1234 = 6 символа + '\0' = 7 байта — buffer overflow!
[OK]  char id[7];    // 6 + 1 за '\0' — правилно
```

СЕКЦИЯ 7 — ЧЕКЛИСТ ПРЕДИ ПРЕДАВАНЕ

Преди да предадеш, провери всяка точка. Пропускането на която и да е коства точки.

```
ЧЕКЛИСТ — 14 ТОЧКИ ПРЕДИ ПРЕДАВАНЕ НА ИЗПИТА

  [ ]  char[] в структурата, НЕ char*          (за fwrite/fread да работят)
  [ ]  char масивите имат +1 за '\0'           (AA1234 -> char id[7])
  [ ]  arr = realloc(...)                       (не само realloc без присвояване)
  [ ]  NULL проверка след всеки realloc         (if (!arr) exit(1))
  [ ]  addElement() е Member* (не void!)        (съдържа return arr;)
  [ ]  arr = addMember(arr, &count) в main()   (не само addMember(...))
  [ ]  NULL проверка след всеки fopen           (if (!f) return;)
  [ ]  fclose() след всяка работа с файл
  [ ]  Бинарни файлове с 'rb' / 'wb'           (не 'r' / 'w')
  [ ]  strcmp() за сравнение на низове          (не ==)
  [ ]  strcpy() за копиране на низове           (не =)
  [ ]  scanf за числа — СЪС &                   (scanf("%f", &price))
  [ ]  scanf за низове — БЕЗ &                  (scanf("%54s", names))
  [ ]  free(arr) в края на main()

  Ако всички 14 точки са ОК -> предавай!
```

Успех на изпита!